
SOFTWARE ENGINEERING

NOTEBOOK

PRODUCED BY: *vaeVictis*

leo.expat@gmail.com

SEE SECTION ?? FOR SOCIAL CONTACTS.

THIS DOCUMENT IS WRITTEN IN L^AT_EX

ENJOY!

Table of Contents

1	Brief Introduction to Latex	3
1.1	Downloading Tex Studio	3
1.2	Source Code and Resume Templates	3
1.3	LaTeX Files	3
1.3.1	LaTeX Commands	4
1.4	General Items	4
1.5	Terminology to Know	4
1.6	Tex Studio Commands	4
1.7	Creating Templates	4
2	Solid Principles	6
2.1	Single-Responsibility	6
2.2	Open-Close	6
2.3	Liskov Substitution	6
2.4	Interface Segregation	7
2.5	Dependency Inversion	7
3	Design Patterns	8
3.1	Creational Patterns	8
3.1.1	Factory Method	8
3.1.2	Abstract Factory	10
3.1.3	Builder	11
3.1.4	Prototype	12
3.1.5	Singleton	13
3.2	Behavioral Patterns	14
3.2.1	Chain of Responsibility	14
3.2.2	Command	14
3.2.3	Iterator	15
3.2.4	Mediator	16
3.2.5	Memento	17
3.2.6	Observer	18
3.2.7	State	19
3.2.8	Strategy	20
3.2.9	Template Method	21
3.2.10	Visitor	22
3.3	Structural Patterns	24
3.3.1	Adapter	24
3.3.2	Bridge	25

3.3.3	Composite	26
3.3.4	Decorator	26
3.3.5	Facade	27
3.3.6	Flyweight	28
3.3.7	Proxy	29
4	Smart Pointers	31
4.1	Smart Pointers in C++	31
4.1.1	std::unique_ptr	31
4.1.2	std::shared_ptr	31
4.1.3	std::weak_ptr	31
5	References	33

About This File

This file was created for the benefit of all teachers and students wanting to use Latex for tests/exams/lessons/thesis/articles etc.

The entirety of the contents within this file, and folder, are free for public use.

Brief Introduction to Latex

LaTeX, which is pronounced «Lah-tech» or «Lay-tech», is a document preparation system for high-quality typesetting. It is most often used for medium-to-large technical or scientific documents but it can be used for almost any form of publishing.

When using Latex, ***make sure to keep your source code organized by indenting and using sections/chapters/subsections.*** Not keeping your source code organized makes it harder to fix possible errors in your code.

1.1 Downloading Tex Studio

Latex is free to download and is available on all types of operating systems. To download $\text{\LaTeX}$, click the url to the right: <https://www.texstudio.org/>

1.2 Source Code and Resume Templates

If interested in obtaining the source code for this guide, see:

<https://sourceforge.net/p/latex-source-code/wiki/Download/>

If interested in obtaining some of the resume templates I have created, see:

<https://sourceforge.net/p/latex-resume-template/wiki/download/>

1.3 LaTeX Files

LaTeX files have names that end with the extension `.tex`: for example, an acceptable file name might be `myfile.tex`. (Never use spaces in file names.) The input file contains both the text of

your document and the LATEX commands needed to format it. The first command in the file, `\documentclass`, defines the style of the document.

1.3.1 LaTeX Commands

To distinguish them from text, all LATEX commands (also called control sequences) start with a backslash `\`. A command name consists of letters only and is ended by a space or a non letter.

1.4 General Items

`\usepackage`

Packages in latex allow for the user to include cool and unique features into ones document. Most packages for latex will be found on www.ctan.org

```
\usepackage{package}
```

1.5 Terminology to Know

- Preamble: All the code that precedes your `\begin{document}`. The preamble section of your document is where all the formatting takes place.
- Commands: Commands are special words that determine $\text{\LaTeX}$ behavior.
- Environments: Environments are used to format blocks of text in a $\text{\LaTeX}$ documents

1.6 Tex Studio Commands

- The button with the magnifying glass, labeled view, in the top part of Tex Studio allows you to view your document sidebyside with your code.
- The green arrows next to the magnifying class, labeled compile, allows you to update your document with any new code you include. Compiling also allows you to check if there are any errors in your code.

1.7 Creating Templates

One of the many great things about $\text{\LaTeX}$ is that offers many ways to avoid some of the repetitive things that come with creating several documents. One of the features $\text{\LaTeX}$ offers is to create personal user templates for files that a user often uses but does not want to continuously type out every time.

For example, lets say you wanted to save a general layout of a lesson document that included **both** the preamble and some of the actual text and commands in the document. To create the template, one would first proceed to the folder originally containing this pdf (The Contents You Downloaded from the Zip File)) and then go to the folder labeled **Files to make Templates out of**. Once there, click the file named: **Lesson Template**. Once you have opened that file in texstudio, go to **File**, then **Make Template**. Once that is done, give the template a name and then press ok.

After creating a template, all a user needs to do to use that template is to go back to **File**, and then **New From Template** and then click the template you created.

Solid Principles

SOLID is a mnemonic acronym for five design principles intended to make object-oriented designs more understandable, flexible, and maintainable.

The principles are a subset of many principles promoted by American software engineer and instructor **Robert C. Martin**, [1][2][3].

They were first introduced in his 2000 paper Design Principles and Design Patterns, while the acronym was introduced later, around 2004, by **Michael Feathers**.

Although the SOLID principles apply to any object-oriented design, they can also form a core philosophy for methodologies such as agile development or adaptive software development.

2.1 Single-Responsibility

"There should never be more than one reason for a class to change." [5]

In other words, every class should have only one responsibility. [6]

2.2 Open-Close

"Software entities ... should be open for extension, but closed for modification." [7]

2.3 Liskov Substitution

"Functions that use pointers or references to base classes must be able to use objects of derived classes without knowing it." [8]

See also: https://en.wikipedia.org/wiki/Design_by_contract

2.4 Interface Segregation

"Clients should not be forced to depend upon interfaces that they do not use." [\[9\]](#) [\[4\]](#)

2.5 Dependency Inversion

"Depend upon abstractions, [not] concretions." [\[10\]](#) [\[4\]](#)

3

SECTION

Design Patterns

Design Patterns describe how to solve recurring design problems to design flexible and reusable object-oriented software, that is, objects that are easier to implement, change, test, and reuse.

3.1 Creational Patterns

3.1.1 Factory Method

Factory Method uses factory methods to deal with the problem of **creating objects without having to specify the exact class** of the object that will be created.

This is done by creating objects by calling a factory method — either specified in an interface and implemented by child classes, or implemented in a base class and optionally overridden by derived classes — rather than by calling a constructor.

When to use this pattern

The object's creation may lead to a significant duplication of code, may require information not accessible to the composing object, may not provide a sufficient level of abstraction, or may otherwise not be part of the composing object's concerns.

The factory method design pattern handles these problems by defining a **separate method for creating the objects**, which subclasses can then override to specify the derived type of product that will be created.

Following a list of use cases

- the exact types and dependencies of the objects your code should work with are not known

- there is the need to provide users of a library or framework with a way to extend its internal components
- there is the concern to save system resources by reusing existing objects instead of re-building them each time.

Structure

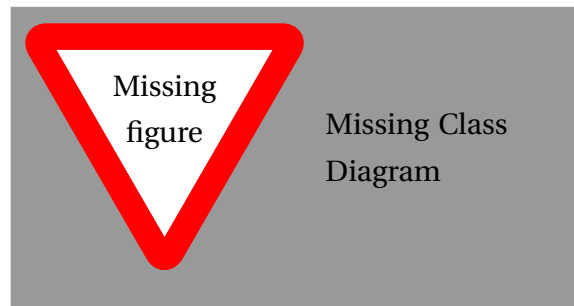


Figure 1: *Factory Method Class Diagram*

The structure of the Factory Method is quite easy

- **Product** defines the interface common to all the objects.
- **Concrete Products** are the different implementations of the previous interface.
- **Creator** is the base class which declares the factory method.
 - **factory method** should either be abstract or return a default product.
 - **returned type** must match the interface
 -
- **Concrete Creators** override the base factory method in order to return different types of **Concrete Products**, which could not be a new instance every time Factory methods could also return existing objects from a cache, an object in a pool or similar ones.

Factory Method helps decouple the creation of the different concrete objects from the core business logic the Creator might and usually does have.

Pros and Cons

Relation with other Design Patterns

3.1.2 Abstract Factory

Abstract Factory makes easier to produce families of related items without specifying their concrete classes.

This is achieved declaring **explicit interfaces for each distinct product** of the product family and making **all variants of products need to follow those interfaces**.

An Abstract Factory is finally declared with a **list of creation methods** for all the items that are part of the product family. For each variant of a product family, **a separate factory class** based on the AbstractFactory interface will be needed and **clients access with both factories and products via their abstract interfaces**.

When to use this pattern

-
-
-

Structure

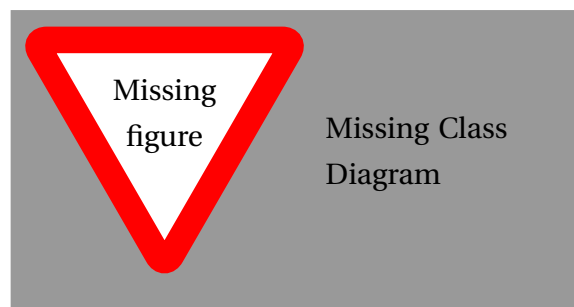


Figure 2: *Abstract Factory Class Diagram*

- **Abstract Products** declare the interfaces for a set or family of related but distinct products.
- **Concrete Products** are the implementations of the concrete variants of abstract products. Each single abstract product must be implemented in all given variants.
- **Abstract Factory** is the interface to collect the declarations of sets of methods to create each of the abstract products.
- **Concrete Factorys** implement the creation methods of the abstract factory, each creating only the product variant it corresponds to.

- **Client** can work with any concrete factory as long as it communicates with the objects through their abstract interfaces. In order to have this behavior, the signatures of the concrete factories' creation methods must match the corresponding abstract interfaces'.

Pros and Cons

Relation with other Design Patterns

3.1.3 Builder

Builder facilitates the **construction of complex objects step by step**, producing different types and representations of an object using the same construction code. This is achieved extracting the construction code out of its own class and moving it to a separate Builder class. Construction is managed with a set of steps executed in series and only in case they are necessary for producing a specific configuration of the object. A specific Builder class could be implemented to manage the same building steps in a different manner. A Director class, not mandatory, can be implemented to collect the steps in a specific order, while the Builder provides the implementation of what to do.

When to use this pattern

-
-
-

Structure

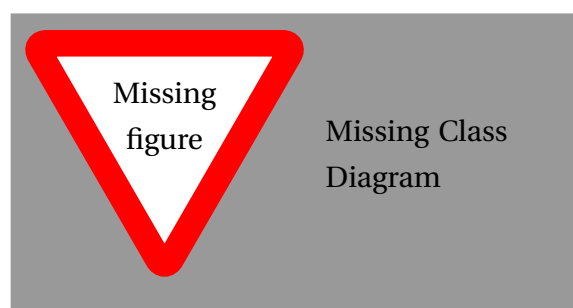


Figure 3: *Builder Class Diagram*

- **Builder** interface declares the production steps common to all the builders.
- **Concrete Builders** implement the construction steps needed for products which do not follow the common interface.
- **Products** are the resulting objects constructed by different builders.

- **Director** might be useful to call the steps needed
- **Client** associates a builder with a director in order to let latter be used for any further construction.

Pros and Cons

Relation with other Design Patterns

3.1.4 Prototype

Prototype enables the copy of existing objects without implementing code which depends on their classes. This is achieved delegating the cloning process to the actual objects that are being cloned. A common interface is implemented for all the objects that support cloning. The clone method is similar for all classes: it creates an object of the current class and carries over all the fields values of the old cloned one. Copying private fields is possible for any programming language which allows to access private fields of other objects of the same class. An object which could be cloned is called Prototype. Cloning is an alternative to subclassing for objects with many fields and configurations.

When to use this pattern

-
-
-

Structure

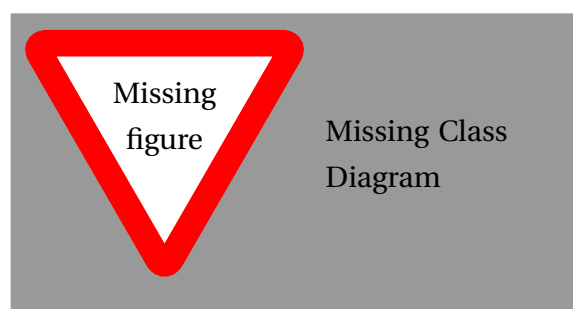


Figure 4: *Prototype Class Diagram*

- **Prototype** interface declares the cloning method
- **Concrete Prototype** implements the cloning method and may also handle some logic needed for edge cases of the cloning process.
- **Client** can produce a copy of any object which follows the prototype interface.

Pros and Cons

Relation with other Design Patterns

3.1.5 Singleton

Singleton ensures that a class could have one instance only, while providing a global access point to this instance. This functionality could be interpreted as a violation of the Single Responsibility Principle.

When to use this pattern

-
-
-

Structure

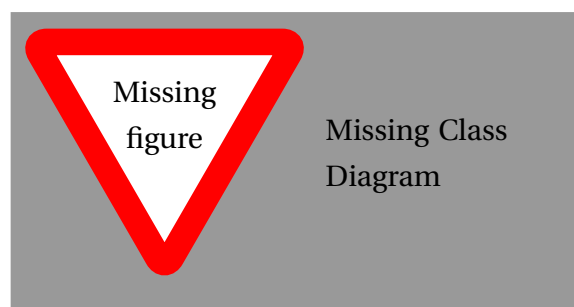


Figure 5: *Singleton Class Diagram*

- **Singleton** class should have a private constructor, in order to prevent the client from calling it. A static method `getInstance` is responsible to create and store the first and only instance in a static field. Successive calls return the same instance.

Pros and Cons

Relation with other Design Patterns

3.2 Behavioral Patterns

3.2.1 Chain of Responsibility

Chain of Responsibility makes possible to pass a request along a chain of handlers, each of which decides upon receiving a request either to process the request or to pass it to the next handler in the chain.

When to use this pattern

-
-
-

Structure

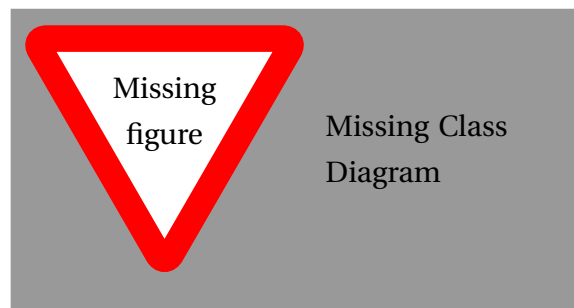


Figure 6: *Chain of Responsibility Class Diagram*

- **Handler** declares a common interface for all the concrete handlers, usually in terms of a first method for handling the requests and second one to set the next handler in the chain.
- **Base Handler** could be optionally implemented
- **Concrete Handlers** contain the actual code to process the request, which will be executed after an optional preliminary check to understand whether it could be processed or passed on.
- **Client** may create chains both statically and dynamically, and depending on its own logic as well. The request can be sent to any handler in the chain.

Pros and Cons

Relation with other Design Patterns

3.2.2 Command

Command

When to use this pattern

-
-
-

Structure

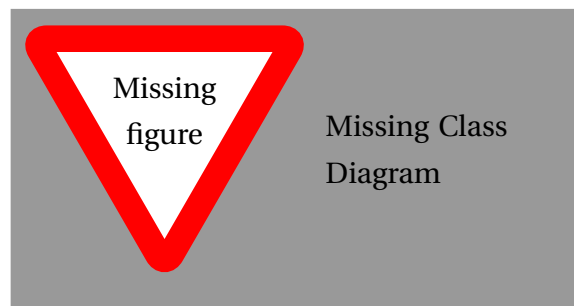


Figure 7: *Command Class Diagram*

- **Sender or Invoker** is responsible for creating and sending requests. For this purpose, the class needs a field to store a reference to a command object. Instead of sending the request directly to the receiver, the sender directly triggers a command objects, which are usually provided and not created by the sender itself.
- **Command** interface declares the method to execute the command.
- **Concrete Command** is meant to implement various kind of requests, passing the call to one of the business logic objects. It is possible to store the parameters needed to execute the command in a field of the class.
- **Receiver** implements some business logic and most of the time does the actual work, while commands only handle the details of how a request is passed to the receiver.
- **Client** creates and configures commands, passing all the parameters needed and a reference to the Receiver.

Pros and Cons

Relation with other Design Patterns

3.2.3 Iterator

Iterator enables the client to traverse elements of a collection without exposing its underlying representation. This is achieved extracting the traversal behavior of a collection into a separate object, called iterator. Details of the traversals are encapsulated into the iterator class; in this

way several iterators can go through the same collection at the same time and independently of each other. Iterators should implement the same interface in order to make the client code compatible with any collection type or any traversal algorithm, as long as there's a proper iterator.

When to use this pattern

-
-
-

Structure

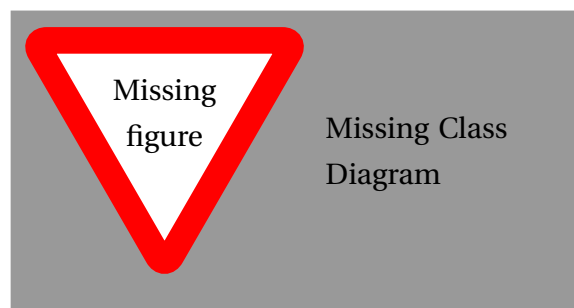


Figure 8: *Iterator Class Diagram*

- **Iterator** interface declares the operations needed to traverse a collection, like fetching the next element, keeping track of the current position and so on.
- **Concrete Iterator** implements specific algorithms each independently on others in order to be able to use many of them at the same time.
- **Collection** interface declares one or multiple methods for getting iterator compatible with the collection. The return type of these methods must match the Iterator interface.
- **Concrete Collection** returns instances of a particular concrete iterator class on client's request.
- **Client** can work with both the collection and the iterator using their interfaces.

Pros and Cons

Relation with other Design Patterns

3.2.4 Mediator

Mediator reduces chaotic and confusing dependencies between objects, restricting direct communications between the objects and forcing them to collaborate only through a mediator object. This is achieved cutting all direct communication between the components which are required to be independent of each other and ...

When to use this pattern

-
-
-

Structure

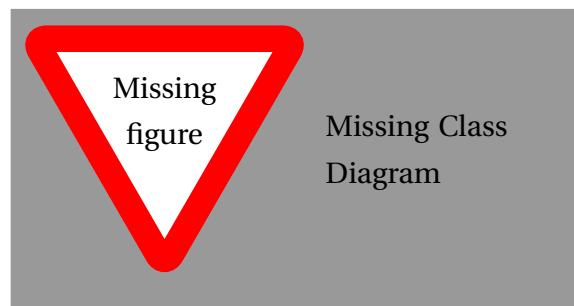


Figure 9: *Mediator Class Diagram*

- **Components** are classes containing some business logic. Each one has a reference to a mediator, whose referenced type is the Mediator interface's. Components are unaware of the actual Mediator class, so that it could be reused in other contexts with different mediators.
- **Mediator** interface declares the communication structure between components. Components may pass any context as argument of their methods, although they must avoid coupling the receiving component with the sender.
- **Concrete Mediator** implement relation between various components. It could be useful to keep references to all the components managed and their lifecycle as well.

Components are not aware of each other. In case some communication needs to go from a Component to a different one, it must pass through the Mediator. Likewise, the sender doesn't know who will handle the request as well as the receiver doesn't know who sent the request.

Pros and Cons

Relation with other Design Patterns

3.2.5 Memento

Memento

When to use this pattern

-
-
-

Structure

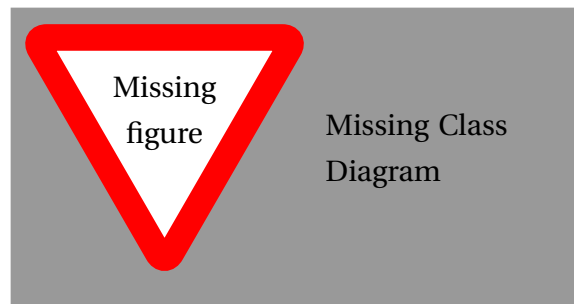


Figure 10: *Factory Method Class Diagram*

- **Originator** is a generic class which could produce snapshots of its own state, as well as restore its state from a snapshot already available.
- **Memento** is a class which acts like a shapshot. It could be made immutable and pass the data only via the constructor and once.
- **Caretaker** knows the "why" and "when" to capture a snapshot or restore a previous one. This class could manage an history of Originator's' states in a stack data structure.

In the previous design, the Memento class is embedded in the Originator, in order to make it easier to the latter to access the former's methods and data members.

Pros and Cons

Relation with other Design Patterns

3.2.6 Observer

Observer

When to use this pattern

-
-
-

Structure

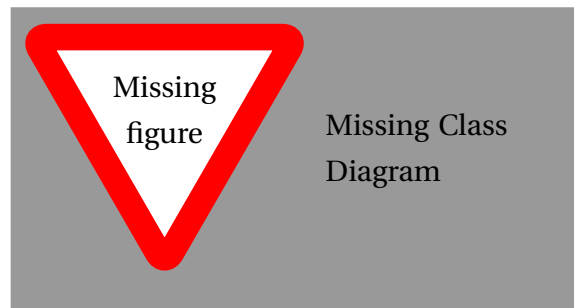


Figure 11: *Observer Class Diagram*

- **Publisher** is a class which issues events of interests to other objects. This class contains a subscription infrastructure that lets subscribers both join and leave the subscriber list.
- **Subscriber** interface declares the notification interface, which in most cases is a single update method with some contextual parameters.
- **Concrete Subscribers** implement the different logics in response to notifications issued by the publisher. As a contextual parameter, a Concrete Publisher
- **Client** creates both publisher and subscriber objects and manage the registrations for updates.

Pros and Cons

Relation with other Design Patterns

3.2.7 State

State

When to use this pattern

-
-
-

Structure

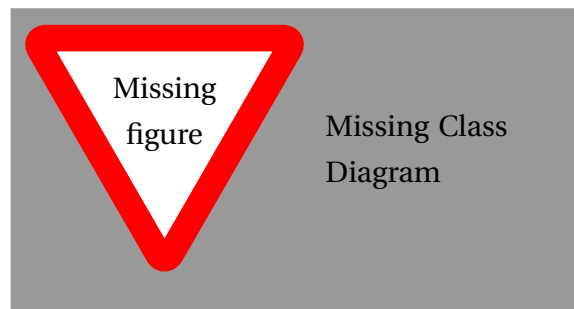


Figure 12: *State Class Diagram*

- **Context** stores a reference to one of the concrete state objects, to which it delegates all the state-related work. The Context interacts with the state interface, in order to be able to interact with any Concrete State.
- **State** interface declares the state generic methods, which should make sense for all concrete states for a matter of consistency.
- **Concrete States** implement their own specific methods. Intermediate abstract classes might help encapsulating some common behavior in order not to duplicate code.

A different Concrete State can be set by both the concrete states themselves.

Pros and Cons

Relation with other Design Patterns

3.2.8 Strategy

Strategy

When to use this pattern

-
-
-

Structure

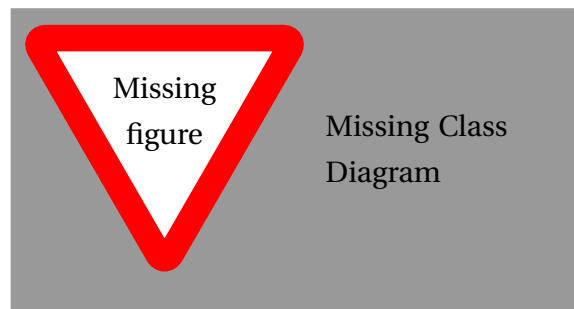


Figure 13: *Strategy Class Diagram*

- **Context** has a reference to the Strategy Interface, used to reference the Concrete Strategy in use, and methods to replace the strategy at run-time.
- **Strategy** interface define the part which is common to all concrete strategies and declares a method used by the Context to execute it.
- **Concrete Strategies** implement a variety of algorithms needed by the context.
- **Client** manages the creation of the strategies and pass them to the context.

Pros and Cons

Relation with other Design Patterns

3.2.9 Template Method

Template Method

When to use this pattern

-
-
-

Structure

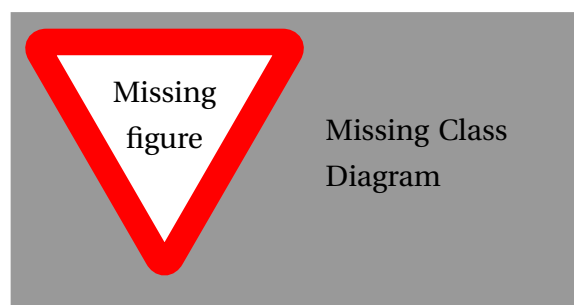


Figure 14: *Template Method Class Diagram*

- **Abstract Class** declares the steps of an algorithm and the **templateMethod** used to call them in a specific order. The steps may both be declared as abstract or implement some actual routines.
- **Concrete Classes** override all of the steps, but not the **templateMethod** itself.

Pros and Cons

Relation with other Design Patterns

3.2.10 Visitor

Visitor

When to use this pattern

-
-
-

Structure

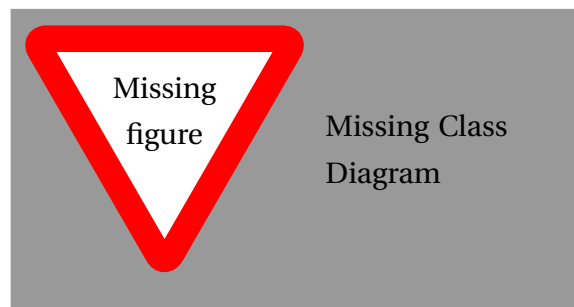


Figure 15: *Visitor Class Diagram*

- **Visitor** interface declares a set of visiting methods, each with a different Concrete Element taken as argument.
- **Concrete Visitor** implements the different behaviors for each Concrete Element.
- **Element** interface declares a method to accept the Visitor interface as argument.
- **Concrete Element** implements the acceptance method, whose purpose is the redirection the call to the proper visitor's method for the current element class. This method must be overridden in any subclass even though it was already developed in the base one.
- **Client** usually needs to work on a collection or a similar complex object whose type is ignored by the Client, since it works with the abstract interface.

Pros and Cons

Relation with other Design Patterns

3.3 Structural Patterns

3.3.1 Adapter

Adapter allows the collaboration among objects with incompatible interfaces. This is achieved through the implementation of an Adapter which make one interface understandable by an object of a different class. The adapter gets an interface compatible with one existing object. Through this interface, the objects can call the adapter's methods. Upon receiving a call, the adapter passes the request to the second object, in a format and order it could understand. An Adapter could work in both directions.

When to use this pattern

-
-
-

Structure

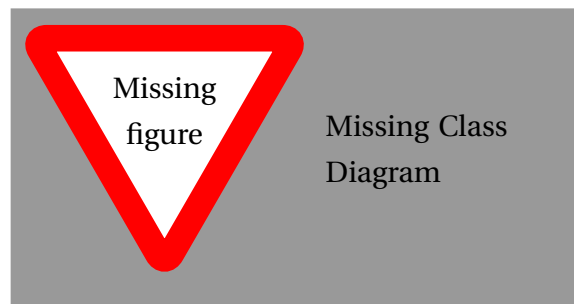


Figure 16: *Adapter Class Diagram*

- **Client** is the business logic of the program.
- **Client Interface** defines the protocol for a class to be able to collaborate with it.
- **Service** is usually a 3rd party or legacy software with an interface incompatible with the Client.
- **Adapter** is a class able to work with both the client and the service. It receives calls from the client and translates into calls to a wrapped service object.

Since the Client interacts with the adapter through the client interface, it doesn't get coupled. Therefore, different adapters can be added to the program without breaking any existing code.

Pros and Cons

Relation with other Design Patterns

3.3.2 Bridge

Bridge splits a large class or a set of closely related classes into two separate hierarchies which can be developed independently of each other. This is achieved by switching from inheritance to object composition. One of the dimensions is extracted into a separate class hierarchy, therefore the original classes will reference an object of the new hierarchy instead of having all its states and behaviors coded within the class. The original class implement a reference field for each "extracted-dimension" classes This extraction makes a class able to delegate any 'dimension-related' work to the linked 'dimension-related' object.

When to use this pattern

-
-
-

Structure

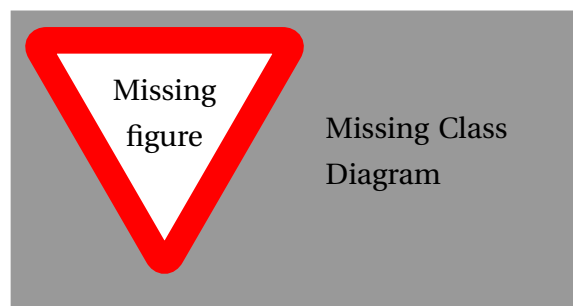


Figure 17: *Bridge Class Diagram*

- **Abstraction** provides high-level interface, and relies on the Implementation objects to perform the actual work.
- **Implementation** is the interface shared by all the concrete implementations. Usually the abstraction declares some complex behavior that relies on primitive operations declared by the implementation.
- **Concrete Implementations** implement specific code.
- **Refined Abstraction** are optional and might help providing variants of control logic.
- **Client** interacts with the abstraction, after linking it with one of the implementation objects.

Pros and Cons

Relation with other Design Patterns

3.3.3 Composite

Composite after enabling composition of objects into tree structures, this pattern let clients work with these structures as they were individual objects. This is achieved extending the class

When to use this pattern

-
-
-

Structure

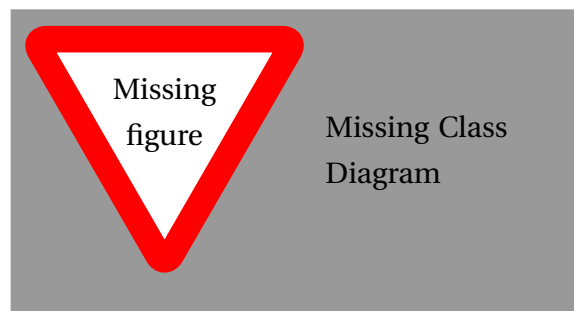


Figure 18: *Composite Class Diagram*

- **Component** interface contains the common operations for both simple and complex elements of the tree.
- **Leaf** is a basic element of a tree and doesn't contain any other sub-element. Usually is the part of the system which ends up doing the work.
- **Container** or Composite is an element that has sub-elements, which means leafs and containers as well. It interacts with sub-elements through their interfaces.
- **Client** can work with both simple and complex elements through the component interface.

Pros and Cons

Relation with other Design Patterns

3.3.4 Decorator

Decorator attaches new behaviors to objects by placing these objects inside special wrapper objects containing the desired behavior.

When to use this pattern

-
-
-

Structure

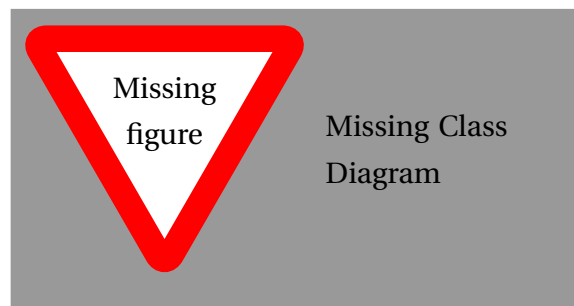


Figure 19: *Decorator Class Diagram*

- **Component** is the interface common for both wrapper and wrapped objects.
- **Concrete Component** is a class being wrapped and whose basic behavior can be altered by decorators.
- **Base Decorator** has a reference of the wrapped object, whose referenced type matches the Component interface. The Base Decorator delegates all operations to the wrapped object.
- **Concrete Decorators** define behaviors which can be added to Components dynamically. The concrete decorator overrides the base behavior and execute the variant either before of after calling the parent method.

Multiple layers of decorations are possible as long as the Client interacts with the object through the component.

Pros and Cons

Relation with other Design Patterns

3.3.5 Facade

Facade consists of a simplified interface to a complex subsystem which contains lots of moving parts. Functionalities provided by the Facade could be reduced to the subset the client is interested in.

When to use this pattern

-
-
-

Structure

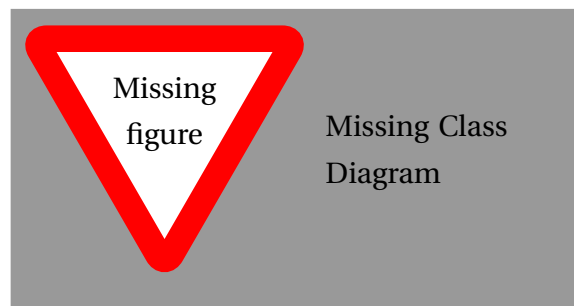


Figure 20: *Facade Class Diagram*

- **Facade** implements a convenient access to a particular part of the subsystem's functionality and knows where to direct the client's request and how to operate all the system's parts.
- **Additional Facade** might help reducing the tasks managed by a single Facade. It can be used by the Client and by the other facades as well.
- **Complex Subsystem** Subsystem class are various object which need to be studied deeply in order to manage them properly. These classes work together and are unaware of the Facade's existence.
- **Client** calls the subsystem classes through the Facade only.

Pros and Cons

Relation with other Design Patterns

3.3.6 Flyweight

Flyweight reduces RAM consumption by sharing common parts of state between multiple objects, instead of creating copies of same data in each object.

When to use this pattern

-

-
-

Structure

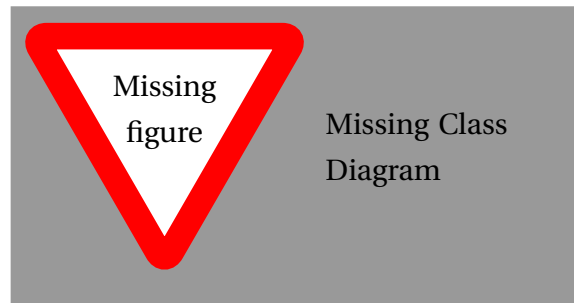


Figure 21: *Flyweight Class Diagram*

- **Flyweight** contains the portion of the original object's state that can be shared between multiple objects. The state stored inside a flyweight is called ***intrinsic***; the state passed to the flyweight's methods is called ***extrinsic***. The same flyweight object can be used in many contexts.
- **Context** contains the extrinsic state, unique across all original objects. It represents the full state when paired with one of the flyweight objects.
- **Flyweight Factory** manages a pool of existing flyweights freeing the Client from the task of creating. The Flyweight Factory receives a call to retrieve a specific flyweight, looks over previously created flyweight and either returns one of these or creates a new one in case nothing matches the search criteria.
- **Client** sees the flyweight as a template object which can be configured at runtime by passing some contextual data into parameters of its methods.

Pros and Cons

Relation with other Design Patterns

3.3.7 Proxy

Proxy

When to use this pattern

-
-
-

Structure

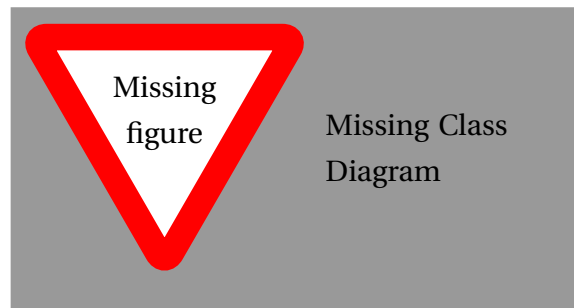


Figure 22: *Proxy Class Diagram*

- **Service Interface** declares the interface to the Service, which must be followed by the Proxy in order to be able to disguise itself as a service object.
- **Service** is a class whose business logic is proxied.
- **Proxy** references a service object and manages to process requests and pass them on to the service object, whose lifecycle sometimes could be managed by the Proxy as well.
- **Client** can work with a Proxy until the latter's interface is the same as the Service's.

Pros and Cons

Relation with other Design Patterns

4

SECTION

Smart Pointers

Generally speaking, a **smart pointer** is a data type which can be used like a pointer, but which provides the additional feature of memory management which frees the memory when the smart pointer is no longer used.

4.1 Smart Pointers in C++

4.1.1 `std::unique_ptr`

This specific pointer can be used when you want your object to live just as long as a single owning reference to it lives. For example, use it for a pointer to memory which gets allocated on entering some scope and de-allocated on exiting the scope

4.1.2 `std::shared_ptr`

4.1.3 `std::weak_ptr`

4

SECTION

References

- [1] Robert C. Martin (2014). *Principles of OOD*
Retrieved from: <http://butunclebob.com/ArticleS.UncleBob.PrinciplesOfOod>
- [2] Robert C. Martin (2009). *Getting a SOLID start*
Retrieved from: <https://web.archive.org/web/20130917122741/https://sites.google.com/site/unclebobconsultingllc/getting-a-solid-start>
- [3] Sandu Metz (2009). *SOLID Object-Oriented Design*
Retrieved from:
<https://ghostarchive.org/varchive/v-2yFMzxqwU>
- [4] Robert C. Martin (2000). *Design Principles and Design Patterns*
Retrieved from:
https://web.archive.org/web/20150906155800/http://www.objectmentor.com/resources/articles/Principles_and_Patterns.pdf
- [5] Robert C. Martin (2015) *Single Responsibility Principle*
Retrieved from:
<https://web.archive.org/web/20150202200348/http://www.objectmentor.com/resources/articles/srp.pdf>
- [6] Robert C. Martin (2003) *Agile Software Development, Principles, Patterns, and Practices*
Retrieved from:
<https://web.archive.org/web/20150202200348/http://www.objectmentor.com/resources/articles/srp.pdf>

- [7] *Open/Closed Principle*
Retrieved from:
<https://web.archive.org/web/20150905081105/http://www.objectmentor.com/resources/articles/ocp.pdf>
- [8] *Liskov Substitution Principle*
Retrieved from:
<https://web.archive.org/web/20150905081111/http://www.objectmentor.com/resources/articles/lsp.pdf>
- [9] *Interface Segregation Principle*
Retrieved from:
<https://web.archive.org/web/20150905081110/http://www.objectmentor.com/resources/articles/isp.pdf>
- [10] *Dependency Inversion Principle*
Retrieved from:
<https://web.archive.org/web/20150905081103/http://www.objectmentor.com/resources/articles/dip.pdf>
- [11] *Clean Architecture: A Craftman's Guide to Software Structure and Design*
Retrieved from:
<https://books.google.com/books?id=uGE1DwAAQBAJ&q=2004+or+thereabouts+by+Michael+Feathers>
- [12]
Retrieved from: